

EDEREST

EdTech Hack&Hire · 3-Day Project Brief

Mastercard Foundation EdTech Fellowship · 3 working days · Team of 4 participants · 22-24 September · Agreed scope, Day 1

1. Startup Overview

Ederest runs two products on one stack: an LMS/Academy web application delivering professional programmes (notably SAP), and a Digital Adoption Platform delivered as a Chromium browser extension. The backend is a Python FastAPI modular monolith on Azure, with a React SPA front-end, PostgreSQL and self-hosted PostHog. AI is already used in production to turn product screenshots into step-by-step tutorials.

2. Hackathon Challenge

Ederest has identified learner data collection as a priority need, starting where learners struggle most: learning to use a business software. The platform captures technical events, but it does not systematically collect where a learner gets stuck on a task, what they think at that moment, and why they give up. That feedback exists only in scattered conversations, and today's exercises assume a mouse, which leaves out learners who cannot use one. The challenge is to build a lightweight, accessible feedback loop around a guided software task: detect the moment a learner is stuck, offer help and a short question at that moment, then turn the raw free-text answers into readable insight for the team.

3. Learning Science Frame

Every team leaves this hackathon with one testable product hypothesis. Before writing code, this team must be able to answer the two questions below, and must revisit them at each daily checkpoint. A working prototype that cannot answer them has not completed the challenge.

Product decision

Learning problem / learner behaviour: Learners get stuck on a step of a software task and abandon it, and the team cannot see which step or why. Learners who cannot use a mouse cannot complete the task at all. The learner behaviour we want to improve is persistence past the blocking step, but first we must be able to locate that step.

Product decision that addresses it: Detect the stuck moment from the learner's behaviour on the step (no progress, repeated wrong clicks, leaving the task), offer an accessible hint and a one-tap question at that exact moment, and compress the free-text answers into a ranked list of blockers the team can act on.

Learning mechanism supported: Just-in-time support at the point of difficulty: the hint helps the learner get past the step without leaving the task. The feedback loop then acts on the content team: its value comes from whether the blockers it surfaces are the real ones, and whether fixing them changes completion.

Evidence of learning

What the team must observe or measure to know whether the decision actually improved learning:

- Response rate per trigger moment: a signal nobody answers is not a signal.
- Diagnostic validity: do the AI-surfaced themes match the steps with the highest measured drop-off?
- Persistence: do more testers get past the blocking step with hints on than with hints off?
- Accessibility: can the whole task, the hint and the survey be completed with the keyboard only?
- Whether a reader of the dashboard can name the single step to fix first, and why, in under 60 seconds.
- Specificity of the themes: 'Step 3 does not say which field to fill' is actionable; 'the software is difficult' is not.

AI in this solution

The AI does not tutor: the hints are written per step, and the AI clusters and summarises what learners report. The honest measure of the AI layer is therefore diagnostic accuracy, whether its themes point at the steps where

learners actually stop, not learner satisfaction.

TESTABLE HYPOTHESIS · AGREED WITH THE STARTUP ON DAY 1

If a learner who is stuck on a step gets an accessible hint and a one-tap question at that exact moment, then more learners will get past that step, and the AI-surfaced themes will identify the same steps as the measured drop-off data, with enough specificity for the team to name a concrete fix.

Measure that could show the team was wrong: the step named by the top AI theme is not among the steps with the highest measured drop-off, or completion past the blocking step does not improve with hints on.

4. Business / User Objective

- Give Ederest a structured, continuous source of learner feedback instead of ad-hoc conversations.
- Make blocking steps visible so guides and content can be fixed where it actually hurts.
- Make the guide usable by every learner, including learners who cannot use a mouse.
- Show how AI can compress dozens of free-text comments into a short list of themes that a small team can act on.
- Collect the team's own product ideas on accessibility and on making software easier to learn.

5. Recommended Team Composition

Team size: 4 participants

- 2 Full-Stack Developers
- 1 AI Engineer
- 1 Digital Marketer

The Full-Stack Developers build the practice screen with its step guide, the embeddable micro-survey component, the collection API and the insight dashboard. The AI Engineer builds the analysis layer that clusters free-text answers into themes with sentiment and suggested actions. The Digital Marketer designs the stuck moments and the survey wording so that response rates are high, defines the KPIs the dashboard must display, and recruits the testers. Every participant contributes two idea cards (see section 7).

6. Proposed MVP

An accessible learner feedback loop made of four connected pieces: a guided practice task on a mock business software screen, where a stuck learner is detected and offered a hint; a small survey widget that appears at defined moments (stuck on a step, leaving the task, end of the task); a collection service that stores answers and session events with the step context; and a dashboard where the Ederest team sees response rates, drop-off and satisfaction per step, and an AI-generated summary of what learners struggle with and what should be fixed first. The guide and the survey work with the keyboard only.

7. Functional Requirements

Must Have

- A guided practice task: a mock business software screen (for example “create a purchase order”) with a 4-step guide showing the current step and a hint.
- Stuck detection that triggers the survey when there is no progress on a step for a set time, after repeated wrong clicks, or when the learner leaves the task, and at the end of the task.
- An embeddable survey widget (one rating question plus one open question) that can be triggered at those moments.
- Keyboard-only use of the guide and the widget (visible focus, Tab, Enter, Escape), with labels for screen readers.

- A collection endpoint that stores the answer with task and step context, trigger type and a timestamp, without storing personal identifiers.
- A dashboard showing response rate per trigger moment, average rating per step and the list of raw comments.
- An AI analysis view that groups comments into three to five themes with a short summary per theme.
- Instrumentation of the learning evidence: drop-off per step is computed from the session events of the practice task, independently of the survey answers, so the AI themes can be checked against it.
- Each generated theme must cite the number of comments behind it and the step(s) concerned, so a claim can be traced back.
- Two idea cards per participant, one on accessibility and one on making a software easier to learn (the moment, the idea, how to test it, how to measure it), included in the handover pack.

Should Have

- Voice command to drive the guide (“next”, “repeat”, “back”) with the browser’s built-in speech recognition.
- Read-aloud of the current step.
- A hints on/off switch, to compare completion with and without hints.
- Sentiment classification per comment (positive / neutral / negative).
- A drop-off view showing at which step learners stop.
- Export of the insight summary as a shareable file.

Could Have / Stretch Goals

- AI-suggested rewrite of the step instruction behind each theme.
- Multi-language handling (French, English, Arabic) in the analysis layer.
- A weekly digest email mock-up.

Changes agreed on Day 1

- The demo learning path is replaced by a guided software task: most Ederest learning is hands-on software practice (SAP), and it is the use case of the DAP extension.
- Trigger moments are detected from behaviour on a step: this is the exact friction moment the brief asks for.
- Accessibility moves into the Must-Haves: a question some learners cannot answer produces biased data.
- Idea cards are added: Ederest may develop the strongest ideas after the event, crediting their authors.

8. Technical Requirements

- Front-end: React + TypeScript for the practice screen, the guide, the widget and the dashboard, in line with the existing stack. The guide and the widget are one self-contained component, so it could later run inside the DAP extension.
- Back-end: a small Python FastAPI service (or equivalent) with two or three endpoints (for example events, responses, insights).
- Database: PostgreSQL or SQLite; four or five tables are sufficient (steps, sessions, events, responses, themes).
- AI: an existing LLM API for clustering and summarisation. No model training.
- Privacy: store an anonymous session identifier only, never names, emails or IP addresses, and remove emails and phone numbers from comments before storage. This is a deliberate design constraint from Ederest.
- Accessibility: a keyboard-only walkthrough of the whole task and an automated audit (Lighthouse or axe) before the demo.
- Data: seed the database with 30-60 realistic sample responses and matching session events so the dashboard is meaningful during the demo.

9. User Journey / Main Workflow

- Input - A learner practises a task on the mock software screen, gets stuck on a step or leaves, and is offered a hint and a two-question survey.
- Process - The answer and the session events are stored anonymously with the step context; the AI layer periodically analyses the collected free text.
- Output - The Ederest team opens the dashboard and sees response rates, drop-off and ratings per step, and three to five themes summarising what learners said.
- User action - The team decides which step, hint or instruction to fix first, based on the ranked themes.

10. 3-Day Execution Plan

Day 1 · Frame, scope and build the foundation (remote)

- Understand the practice task and agree on the stuck moments with the startup.
- Fix the data model and the survey questions (Digital Marketer leads the wording).
- Set up the project, database and the mock software screen with its step guide.
- Build the widget, the stuck detection and the collection endpoint.

Day 2 · Build, integrate and instrument (on site)

- Seed realistic sample data and build the dashboard views.
- Build and test the AI clustering and summarisation layer.
- Connect the dashboard to the AI output.
- Verify end to end: a learner stuck on a step answers the widget and the answer appears in the dashboard. Run the keyboard-only walkthrough.

Day 3 · Finalise, measure and pitch (on site)

- Complete Must-Haves and stop new development by late morning.
- Check the AI summary quality against the seeded data and adjust the prompt.
- Improve dashboard readability, fix bugs and finalise the idea cards.
- Prepare the demo scenario and pitch.

11. Daily Deliverables & Checkpoints

The five checkpoints below are identical for every team in the programme. Their content is specific to this challenge, but their form is not: this is what allows twelve different projects to be reviewed against the same standard at the end of each day. Each item is assessed as delivered, partial or missing.

DAY 1 · submit by 17:00 (remote)

D1.1 Challenge Canvas: One page, in the team's own words: the learning problem, the product decision that addresses it, and the evidence that would show it worked.

D1.2 Testable hypothesis: The hypothesis written out and confirmed with the startup, including the one measure that could show the team was wrong.

D1.3 Scope confirmed: The Must-Have list reviewed with the startup, with anything added or dropped written down and justified.

D1.4 Technical foundation: In place and demonstrable: the mock software screen with its step guide, the data model, the survey wording and the stuck moments agreed with the startup.

D1.5 Stand-up: Two minutes to the coordinator: the hypothesis and the chosen measure, stated out loud.

DAY 2 · submit by 16:30, before the technical dry-run (on site)

D2.1 Working vertical slice: Demonstrable end to end: a learner stuck on a step answers the widget and the answer appears in the dashboard.

D2.2 Evidence instrumented: The measure defined on Day 1 is captured by the product itself: drop-off per step is computed independently, and the AI theme output is stored alongside it for comparison.

D2.3 First measurement: At least one real run of the measure, with the numbers written down; rough is fine, absent is not.

D2.4 Scope and risk log: What will be finished tomorrow, what will not, and what could still break.

D2.5 Dry-run: Three-minute demo of the working slice to the Geeks mentors.

DAY 3 · submit by 12:00, before the pitch session (on site)

D3.1 MVP complete: Against the Must-Have list and the acceptance criteria in this brief.

D3.2 Evidence report: One page: the comparison between AI-surfaced themes and measured drop-off per step across the seeded dataset, the 60-second readability test run on at least 3 people outside the team, and the result of the keyboard-only walkthrough. A result that disproves the hypothesis is a valid and valuable outcome, and must be reported as such rather than hidden.

D3.3 Pitch deck: Ten slides maximum, stating the hypothesis, the evidence collected, and what it would take to test it properly at scale.

D3.4 Handover pack: Repository, README, the open questions left for the startup, and the idea cards.

D3.5 Pitch: Ten minutes plus five minutes of Q&A before the jury.

12. Expected Final Deliverables

Required MVP deliverable

A working prototype where a judge practises the task, gets stuck on a step, answers the widget, then sees the answer appear in the dashboard and be included in the AI-generated theme summary, with the keyboard only if they choose.

Required evidence deliverable

The evidence report described in D3.2, presenting the measure defined on Day 1 and what it showed, including a negative or inconclusive result, which the jury treats as a legitimate outcome when the measurement was honest.

Required ideas deliverable

Two idea cards per participant, in the handover pack.

Optional enhancements

Voice command, read-aloud, hints on/off test, sentiment view, drop-off analysis, export, multi-language handling.

13. Acceptance Criteria

- A survey submitted during the demo is visible in the dashboard within a few seconds.
- The dashboard displays response rate and average rating per step.
- The AI layer produces three to five coherent themes from the collected comments.
- No personal identifier is stored anywhere in the database.
- The whole task, the hint and the survey can be completed with the keyboard only.
- The whole loop can be demonstrated in under three minutes.

- The top AI theme corresponds to a step that the independent drop-off data also flags.
- A reader unfamiliar with the project can name the step to fix first after 60 seconds on the dashboard.

14. What Is NOT Expected

- Integration into the production LMS, the DAP browser extension or PostHog.
- Azure or Kubernetes deployment.
- Authentication and user management.
- Automated test coverage or CI/CD pipelines.
- Production-ready code: the prototype proves the idea, and Ederest's product team rebuilds what is kept.
- Scraping tools, mobile applications or QA process documentation: these are separate Ederest needs, outside this challenge.
- CNDP compliance documentation.

15. Assumptions

- A standalone prototype with a mock software screen is acceptable; no access to production data or code is required.
- Sample learner comments and session events can be generated by the team if the startup cannot provide real ones.
- An LLM API key will be available.
- Anonymous collection is sufficient for the prototype, in line with Ederest's privacy-by-design approach.

16. Questions for the Startup

- At which exact moments in the learning path should the survey appear? **Agreed:** when the learner is stuck on a step (no progress, repeated wrong clicks), leaves the task, or finishes it.
- Can Ederest share anonymised examples of real learner feedback? **Agreed:** the team generates realistic samples, and Ederest reviews them.
- Which KPIs does the team most want to see on the dashboard? **Agreed:** completion per task, drop-off per step, response rate per trigger moment, top blockers.
- Which languages do learners actually write in? **Agreed:** mostly French; the analysis should also handle English and Arabic.
- Should the widget be designed to be embeddable in the DAP extension later, or in the LMS only? **Agreed:** in the DAP extension later; the guide and the widget are built as one self-contained component.